

個人成果物 報告書

項目	内容
システム名	ToDoアプリ(6班)
班番号・氏名	6班 西畑仁太郎
提出日	2026年 8月 7日

本レポートでは、チーム6で開発した ToDo 管理アプリにて、西畑が担当した部分について解説する。西畑の担当は次の2点である。

- A. プロジェクト統合・Git 運用・コード可読性の整備 — チーム全員のコードをまとめ上げる統合役
- B. 締切セクション分け表示機能 — 締切に応じてタスクを自動でグループ分けして見せる機能

1. 担当部分の概要

A. プロジェクト統合・Git 運用・コード可読性の整備

チームは `main` を統合ブランチとし、メンバーごとに作業ブランチ(`JNishihata` / `kimurabranch` / `inchbranch` / `yubbranch`)を分けて開発した。私は基本システムの土台作成(`31f8e98`)に加え、ほぼ全ての Pull Request のマージとコンフリクト解消を担当し、全員の成果物を一つのアプリへ統合し続けた。あわせて、README・CLAUDE.md の整備や、コード全体へのコメント追加・可読性向上(`e2c81ac` ほか)を行い、メンバー全員が読み書きしやすい状態を保った。

B. 締切セクション分け表示機能

タスクを締切の近さに応じて、次の7つのセクションへ自動で振り分けて表示する機能を実装した。

表示	意味	締切までの日数
 期限切れ	締切が過ぎている	0日未満
 今日	締切が今日	0日
 明日	締切が明日	1日
 今週	今週中	2～6日
 来週	来週	7～13日

表示	意味	締切までの日数
 それ以降	それより先	14日以降
∞ 期限なし	締切未設定	—

単なる締切順の並べ替えではなく、「ユーザーが直感的に優先順位を把握できる」ことを狙い、見出し付きのグループ表示にした点が特徴である。関連コミット: [9e9dd24](#) (セクション分けの実装)、[21f43b6](#) (期限切れの強調表示)。

2. コードの工夫点・こだわり

2-1. 締切の「日付だけ」で比較する(時刻の影響を排除)

締切の分類ロジック `getDateSection()` では、今日と締切日の両方の時刻を 0 時にリセットしてから日数差を求めている。これにより「同じ日でも実行した時刻によって判定がずれる」問題を防いでいる。

```
// app.js: getDateSection()
function getDateSection(due) {
  if (!due) return 'no-deadline';

  const today = new Date();
  today.setHours(0, 0, 0, 0); // ← 今日の時刻を 0 時に正規化

  const dueDate = new Date(due);
  dueDate.setHours(0, 0, 0, 0); // ← 締切の時刻も 0 時に正規化

  const diffTime = dueDate - today;
  const diffDays = Math.floor(diffTime / (1000 * 60 * 60 * 24));

  if (diffDays < 0) return 'overdue';
  if (diffDays === 0) return 'today';
  if (diffDays === 1) return 'tomorrow';
  if (diffDays > 1 && diffDays <= 6) return 'this-week';
  if (diffDays > 6 && diffDays <= 13) return 'next-week';
  return 'later';
}
```

こだわり: 締切が未設定 (due が空) の場合を最初に弾き、以降のロジックを日数だけに集中させて読みやすくした。境界(0日・1日・6日・13日)を明示的に書くことで、「今週／来週」の区切りが仕様として一目で分かるようにしている。

2-2. セクションの順序・見出し・アイコンを1か所で管理する

各セクションの「表示順」「ラベル」「アイコン」を `sectionConfig` というオブジェクトに集約した。表示ロジックはこの設定を参照するだけにしてあるため、**セクションの追加・並べ替え・文言変更が、この1か所を直すだけで完結する。**

```
// app.js: セクション定義 (順序・表示名・アイコンを一元管理)
const sectionConfig = {
  'overdue':      { order: 0, label: '期限切れ', icon: '🔴' },
  'today':        { order: 1, label: '今日',      icon: '📅' },
  'tomorrow':     { order: 2, label: '明日',      icon: '📅' },
  'this-week':    { order: 3, label: '今週',      icon: '📅' },
  'next-week':    { order: 4, label: '来週',      icon: '📅' },
  'later':        { order: 5, label: 'それ以降', icon: '📅' },
  'no-deadline':  { order: 6, label: '期限なし', icon: '∞' }
};
```

2-3. 該当タスクが無いセクションは描画しない

`render()` では、まず全タスクをセクションごとに振り分け、その後 `order` の順に並べて見出しとタスクを描画する。**中身が空のセクションは見出しごとスキップ**することで、画面に不要な空欄が並ばないようにした。

```
// app.js: render() のセクション描画部分 (抜粋)
Object.keys(sectionConfig)
  .sort((a, b) => sectionConfig[a].order - sectionConfig[b].order)
  .forEach(sectionKey => {
    if (!sections[sectionKey] || sections[sectionKey].length === 0) return; // 空はス

    const sectionHeader = document.createElement('div');
    sectionHeader.className =
      'task-section-header' + (sectionKey === 'overdue' ? ' overdue-section' : '');
    sectionHeader.textContent =
      `${sectionConfig[sectionKey].icon} ${sectionConfig[sectionKey].label}`;
    listEl.appendChild(sectionHeader);
```

```
const section = document.createElement('div');
section.className = 'task-section';
sections[sectionKey].forEach(({ task, index }) => {
  section.appendChild(createTaskElement(task, index));
});
listEl.appendChild(section);
});
```

2-4. 期限切れをひと目で分かるように強調

期限切れタスクには専用の CSS クラス `overdue` を付け、見出しにも `overdue-section` を付けて、色などで危機感が伝わる見た目にした(21f43b6)。「並び順が先頭」だけでなく「見た目でも警告する」二重の伝達にこだわった。

```
// app.js: createTaskElement() 内 (抜粋)
let className = 'task' + (task.completed ? ' completed' : '');
if (getDateSection(task.due) === 'overdue') {
  className += ' overdue'; // ← 期限切れは専用クラスで強調
}
li.className = className;
```

2-5. (統合役として) 可読性とコメントへのこだわり

複数人が同じファイル(`app.js`)を触るため、「後から読む人が迷わないコメント」を重視した。処理の塊ごとに日本語コメントで意図を書き、`STORAGE_KEY` などの定数には「呼び忘れると変更が消える」といった落とし穴の注意書きまで残している。これはコンフリクトや二重実装を減らすための、統合役としての意図的な工夫である。

3. 課題と解決プロセス

課題1: セクションが重複して表示される不具合(e12749e)

現象: 締切セクション分けを導入した直後、条件によってはタスクや見出しが二重に表示されることがあった。

原因: `render()` はフィルターや操作のたびに一覧を丸ごと作り直す設計だが、描画前に既存の中身をクリアしきれていない／セクションの生成と追加の流れに重複が生じるケースがあった。

解決プロセス:

1. `render()` の最初で一覧要素を毎回まっさらにする (`listEl.innerHTML = ''`) ことを徹底。
2. 「タスクをセクションへ振り分ける処理」と「セクションを描画する処理」を明確に2段階に分離し、描画は `sectionConfig` を1回だけ走査する形に整理。
3. 空セクションのスキップ(2-3)を入れ、余分な要素が生成されないようにした。

結果、重複表示は解消し、テストでも再発しないことを確認した。

課題2: ブランチ統合時のコンフリクト(統合役としての課題)

現象: 複数メンバーが同じ `app.js`, `index.html`, `style.css` を並行して編集していたため、PR をマージする際にコンフリクトが頻発した。

解決プロセス:

1. `main` を唯一の統合ブランチとし、各自の作業ブランチから PR を出す運用に統一 (README にも明記)。
2. マージ時は、各メンバーの変更の意図を読み取り、手作業でコンフリクトを解消。単純な上書きで機能を消さないよう注意した。
3. 関数のまとまりごとにコメントを付け、担当範囲の境界を分かりやすくすることで、そもそもコンフリクトが起きにくいコード構成に近づけた。

4. 他メンバーのコードとの連携

4-1. 既存の描画・データ構造の上に機能を載せた

締切セクション分けは、単独では動かない。他メンバーが実装した以下の仕組みの上に成立している。

- **タスクのデータモデル** `{ title, note, due, priority, genre, completed }`: セクション分けは `due` (締切) を入力として使う。
- **1タスクの生成** `createTaskElement()`: 私はこの関数の出力を、セクションという「入れ物」に振り分けで並べるだけにした。タスク1件の見た目には手を入れず、責務を分けた(期限切れクラスの付与のみ追加)。
- **`render()` の描画ループ**: 既存のフィルター(重要度・ジャンル・検索・完了表示)による絞り込み結果に対して、最後にセクション分けを適用する形にし、**他メンバーの絞り込み機能とそのまま両立**するようにした。

つまり、「絞り込み(他メンバー) → セクション分け(私) → 1件描画(他メンバー)」という流れで、互いの担当が数珠つなぎに連携している。

4-2. 統合役として全員のコードをつなぐ

- 木村・蔭川・谷川の各機能(メモ表示、重要度、ジャンル表示など)を PR 経由で `main` ヘマージし、1つのアプリとして動く状態を維持した。
 - 表示名の対応(重要度の「高／中／低」表記、`getGenreName()` によるジャンルID→名前変換など)が他メンバーの機能と衝突しないよう、命名規則(`taskAddModal` / `genreEditModal` 等)を README に明文化して共有した。
 - 全体のコメント整備・可読性向上により、他メンバーが安心して自分の担当箇所を拡張できる土台を提供した。
-

5. まとめ

私は「締切セクション分け表示」という**ユーザー体験に直結する機能**の実装と、「プロジェクト統合・Git 運用・可読性整備」という**チーム開発を支える役割**の両面を担当した。前者では日付比較の正確さと設定の一元管理にこだわり、後者ではコンフリクト解消とコメント整備を通じて、4人の成果物が1つのアプリとして破綻なく動く状態を保ち続けた。個々の機能開発だけでなく、**それらを1つにまとめ上げる工程**こそが、チーム開発における自分の重要な貢献だったと考えている。